

# Herramientas y arquitectura

Documento técnico del proyecto **wacrm / Agente TED** (versión **0.7.0** en **package.json**, último commit **491d434** del 8-oct-2026). Todo lo que sigue sale del código, de **package.json** / **package-lock.json** y del historial de git.

- Qué hace cada módulo para el usuario → [DOCUMENTACION.md](#)
- Instalación, entorno local y despliegue → [guia-local-y-produccion.md](#)
- API pública → [public-api.md](#)
- Canales → [omnichannel-messenger.md](#), [telegram-setup.md](#), [messenger-troubleshooting.md](#)
- Base de conocimiento → [knowledge-base-sources.md](#)

**Origen del código:** es un *fork* de la plantilla open source [ArnasDon/wacrm](#) (licencia MIT, 474 commits de su autor desde el 16-abr-2026). Sobre esa base hay 8 commits propios ([fpatricioa](#), 6-jul-2026 → 8-oct-2026) que agregan Messenger, Telegram, la ingesta omnicanal, 3 proveedores de IA más, carga de archivos y Google Sheets para la base de conocimiento, derivación a humano con aviso, marca blanca por empresa, la traducción al español y correcciones de difusiones e importación CSV. El remoto actual es [github.com/fraydeveloper/wacrm](#).

## 1. Lenguajes

| Lenguaje                                                            | Dónde                                                                                                                     | Para qué                                                                                                                             |
|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>TypeScript</b><br>( <a href="#">.ts</a> , <a href="#">.tsx</a> ) | <a href="#">src/</a> (356 archivos)                                                                                       | Todo el código de la aplicación: interfaz (React/TSX), rutas de API, lógica de negocio, integraciones y pruebas                      |
| <b>SQL / PL/pgSQL</b>                                               | <a href="#">supabase/migrations/</a> (37 archivos)                                                                        | Tablas, índices, políticas RLS, funciones RPC, <i>triggers</i> y extensiones ( <a href="#">pgvector</a> ) de PostgreSQL              |
| <b>CSS</b>                                                          | <a href="#">src/app/globals.css</a>                                                                                       | Variables de tema (colores, modo claro/oscuro) y estilos base de Tailwind v4                                                         |
| <b>JavaScript</b>                                                   | <a href="#">eslint.config.mjs</a> , <a href="#">postcss.config.mjs</a> , <a href="#">public/opus/encoderWorker.min.js</a> | Configuración de herramientas; el <i>worker</i> minificado de <a href="#">opus-recorder</a> para grabar notas de voz en el navegador |
| <b>YAML</b>                                                         | <a href="#">.github/workflows/ci.yml</a> , <a href="#">.github/dependabot.yml</a>                                         | Integración continua en GitHub Actions y actualizaciones automáticas de dependencias                                                 |
| <b>Markdown</b>                                                     | <a href="#">docs/</a> , <a href="#">README.md</a> , <a href="#">CHANGELOG.md</a>                                          | Documentación                                                                                                                        |

## 2. Frameworks, librerías y plugins

Versiones exactas instaladas según `package-lock.json` (entre paréntesis, el rango declarado en `package.json` cuando difiere).

### 2.1 Servidor y datos

| Paquete                            | Versión            | Para qué sirve en el proyecto                                                                                                                                   |
|------------------------------------|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>next</code>                  | 16.2.6             | Framework: App Router, páginas, rutas de API ( <i>Route Handlers</i> ), middleware, <code>after()</code> para procesar webhooks después de responder            |
| <code>@supabase/supabase-js</code> | 2.108.2 (^2.107.0) | Cliente de PostgreSQL, Auth, Storage y Realtime de Supabase                                                                                                     |
| <code>@supabase/ssr</code>         | 0.12.0             | Sesión de Supabase en cookies para servidor, middleware y navegador                                                                                             |
| <code>pdf-parse</code>             | 2.4.5              | Extrae texto de PDFs subidos a la base de conocimiento ( <code>src/lib/ai/file-extract.ts</code> ); se excluye del <i>bundle</i> en <code>next.config.ts</code> |
| <code>date-fns</code>              | 4.4.0              | Fechas y rangos del panel de métricas                                                                                                                           |

Sin SDK de terceros para Meta, Telegram, Google ni proveedores de IA: todas se llaman con `fetch` directo:

| Servicio externo                          | Archivo                                                                              | Detalle                                                                             |
|-------------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| WhatsApp Cloud API / Messenger            | <code>src/lib/whatsapp/meta-api.ts</code> ,<br><code>src/lib/messenger/api.ts</code> | Graph API v21.0                                                                     |
| Telegram Bot API                          | <code>src/lib/telegram/api.ts</code>                                                 | <code>getMe</code> , <code>setWebhook</code> , <code>sendMessage</code>             |
| Google Sheets API                         | <code>src/lib/google/sheets.ts</code>                                                | JWT de cuenta de servicio firmado a mano con <code>crypto.sign('RSA-SHA256')</code> |
| OpenAI, Anthropic, DeepSeek, Gemini, Z.ai | <code>src/lib/ai/providers/*.ts</code>                                               | Un adaptador por proveedor; DeepSeek y Z.ai usan <code>openai-compatible.ts</code>  |
| Embeddings                                | <code>src/lib/ai/embeddings.ts</code>                                                | Solo OpenAI <code>text-embedding-3-small</code> (1536 dimensiones)                  |

### 2.2 Interfaz

| Paquete                                                      | Versión    | Para qué sirve |
|--------------------------------------------------------------|------------|----------------|
| <code>react</code> / <code>react-dom</code>                  | 19.2.4     | Interfaz       |
| <code>tailwindcss</code> + <code>@tailwindcss/postcss</code> | 4.3.1 (^4) | Estilos        |

| Paquete                                                                     | Versión                      | Para qué sirve                                                                                                                                       |
|-----------------------------------------------------------------------------|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>shadcn</code>                                                         | 4.11.0                       | Generador de componentes base ( <code>components.json</code> , estilo <code>base-nova</code> ); el resultado está en <code>src/components/ui/</code> |
| <code>@base-ui/react</code>                                                 | 1.6.0                        | Primitivas accesibles (diálogos, menús, <i>popover</i> , etc.) que usan los componentes de <code>ui/</code>                                          |
| <code>class-variance-authority</code>                                       | 0.7.1                        | Variantes de componentes (botones, <i>badges</i> )                                                                                                   |
| <code>clsx</code>                                                           | 2.1.1                        | Composición de clases CSS                                                                                                                            |
| <code>tailwind-merge</code>                                                 | 3.6.0                        | Resuelve clases Tailwind en conflicto ( <code>cn()</code> en <code>src/lib/utils.ts</code> )                                                         |
| <code>tw-animate-css</code>                                                 | 1.4.0                        | Animaciones CSS para Tailwind                                                                                                                        |
| <code>lucide-react</code>                                                   | 1.22.0                       | Íconos                                                                                                                                               |
| <code>sonner</code>                                                         | 2.0.7                        | Notificaciones <i>toast</i>                                                                                                                          |
| <code>recharts</code>                                                       | 3.8.1                        | Gráficos del panel (envueltos en <code>src/components/tremor/</code> )                                                                               |
| <code>@xyflow/react</code>                                                  | 12.11.0                      | Lienzo del editor visual de Flujos                                                                                                                   |
| <code>@dagrejs/dagre</code>                                                 | 3.0.0                        | Ordenamiento automático de nodos en el editor de Flujos                                                                                              |
| <code>@dnd-kit/core</code> / <code>sortable</code> / <code>utilities</code> | 6.3.1 /<br>10.0.0 /<br>3.2.2 | Arrastrar y soltar en el tablero Kanban de negocios                                                                                                  |
| <code>opus-recorder</code>                                                  | 8.0.5                        | Grabación de notas de voz en formato Opus desde la bandeja                                                                                           |

## 2.3 Recursos cargados desde fuera (CDN)

No hay etiquetas `<script>` ni hojas de estilo cargadas desde un CDN. Lo único externo es la fuente **Inter** vía `next/font/google` (`src/app/layout.tsx`): Next.js la descarga al compilar y la sirve desde el propio dominio. El *worker* de Opus está copiado en `public/opus/` en lugar de cargarse por CDN.

## 2.4 Base de datos (Supabase)

PostgreSQL gestionado por Supabase, con:

- **Auth** (correo y contraseña) y un *trigger* `handle_new_user` que crea una cuenta y un perfil `owner` en cada registro (migración `017`).
- **RLS** en todas las tablas mediante la función `is_account_member(account_id, min_role)`.
- **Realtime** (`postgres_changes`) para la bandeja en vivo (`src/hooks/use-realtime.ts`).
- **Storage** para avatares, medios del chat y logos.
- **pgvector** (`vector(1536)`) para la búsqueda semántica y `tsvector` para la búsqueda de texto completo de la base de conocimiento (migración `030`).

### 3. Herramientas de desarrollo

| Herramienta                                         | Versión                                          | Uso                                                                                                                                                                                                                       |
|-----------------------------------------------------|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Node.js                                             | <code>&gt;=20.0.0 (engines);</code><br>CI usa 20 | Entorno de ejecución                                                                                                                                                                                                      |
| npm                                                 | lockfile v3                                      | Gestor de dependencias; <code>overrides</code> fuerza versiones parchadas de <code>postcss</code> , <code>ip-address</code> , <code>fast-uri</code> , <code>hono</code> , <code>js-yaml</code> y <code>@babel/core</code> |
| TypeScript                                          | 6.0.3 (^6)                                       | Tipado; <code>npm run typecheck = tsc --noEmit</code>                                                                                                                                                                     |
| Vitest                                              | 4.1.9                                            | Pruebas unitarias ( <code>npm test</code> ); 59 archivos, 606 pruebas                                                                                                                                                     |
| ESLint + <code>eslint-config-next</code>            | 9.39.4 + 16.2.6                                  | Análisis estático ( <code>npm run lint</code> )                                                                                                                                                                           |
| Prettier + <code>prettier-plugin-tailwindcss</code> | 3.9.1 + 0.8.0                                    | Formato ( <code>npm run format</code> )                                                                                                                                                                                   |
| Servidor local                                      | <code>next dev --webpack</code>                  | <code>npm run dev</code> usa webpack porque Turbopack falla en Windows (ver guía); <code>npm run dev:turbo</code> usa Turbopack                                                                                           |
| Git + GitHub                                        | —                                                | Control de versiones; remoto <code>fraydeveloper/wacrm</code>                                                                                                                                                             |
| GitHub Actions                                      | <code>ci.yml</code>                              | En cada <code>push</code> /PR a <code>main</code> : lint → typecheck → test → build                                                                                                                                       |
| Dependabot                                          | <code>dependabot.yml</code>                      | PRs semanales de dependencias (todavía asigna como revisor a <code>ArnasDon</code> , el autor original)                                                                                                                   |

**Estado de las pruebas (8-oct-2026, Windows):** 601 de 606 pasan. Las 5 que fallan (`currency.test.ts`, `date-utils.test.ts`) dependen del idioma y la zona horaria del sistema operativo; es el problema conocido descrito en la guía.

**Hosting:** según la guía, producción corre en **Vercel** con despliegue automático desde `main`. No se usa Apache, por eso no hace falta un `docs/.htaccess`: Next.js solo publica lo que está en `public/`, y `docs/` nunca se sirve en la web.

### 4. Patrón de arquitectura

#### 4.1 Qué patrón es

**No es MVC.** Es una aplicación **Next.js App Router organizada por capas**, con dos caminos hacia los datos:

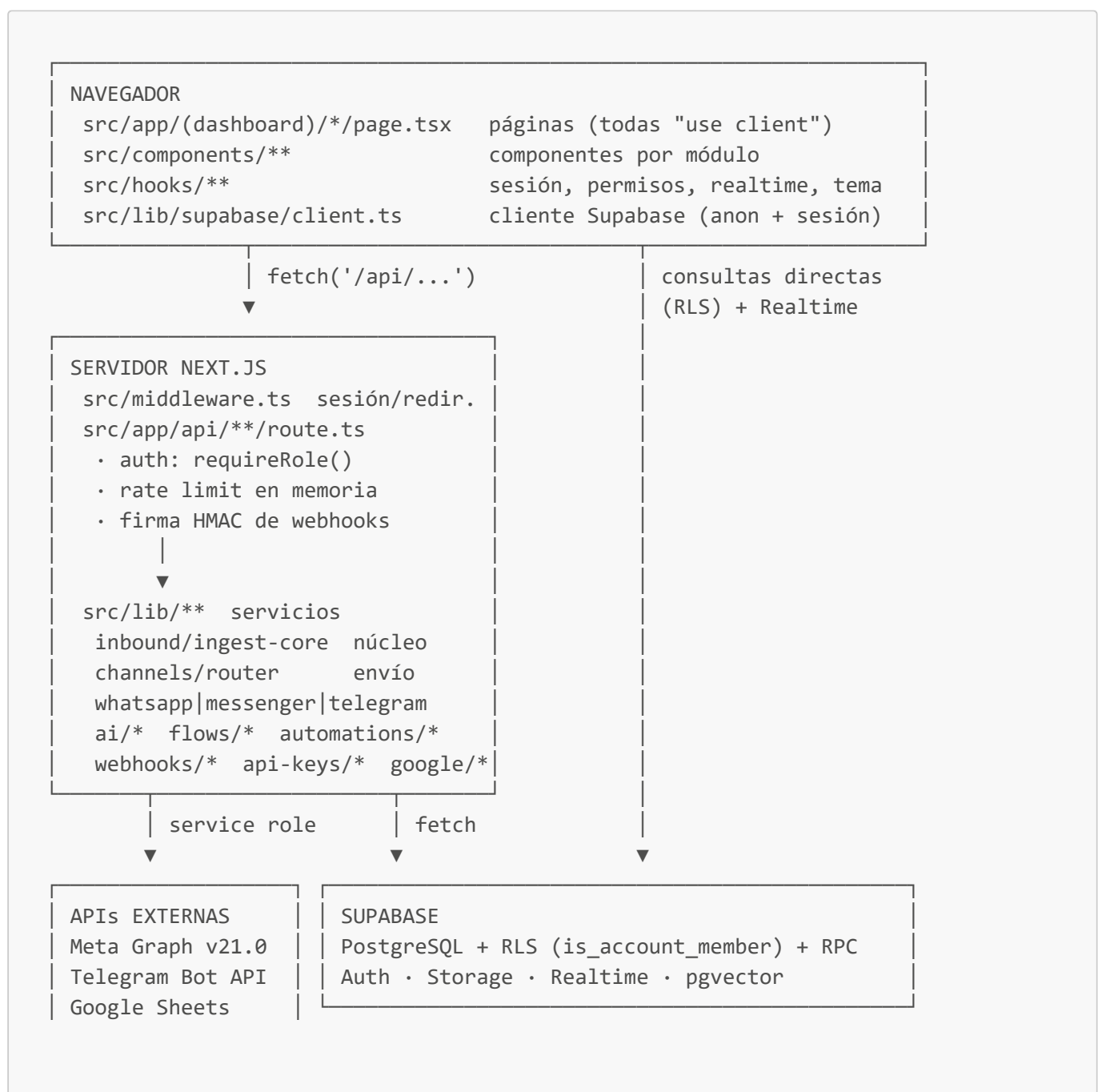
1. **Navegador** → **Supabase directo, protegido por RLS**. Las 17 páginas del panel son componentes de cliente ("`use client`"), y 19 componentes leen y escriben tablas con el cliente de Supabase del navegador. La seguridad no está en un controlador, sino en las políticas RLS de PostgreSQL.
2. **Navegador** → **Route Handler** → **servicios de lib/** → **Supabase/APIs externas**. Las 57 rutas de `src/app/api/` se usan para todo lo que necesita secretos o privilegios: enviar por WhatsApp, cifrar

tokens, llamar a la IA, procesar webhooks, la API pública.

Por qué no encaja en MVC:

- **No hay capa de "Modelo"**: no existen clases de dominio ni ORM. Los tipos de `src/types/index.ts` solo describen filas, y las consultas se escriben con el cliente de Supabase donde se necesitan (en componentes, rutas o `lib/`).
- **Parte de la lógica de negocio vive en la base de datos**: permisos por rol (RLS), contadores atómicos, filtros por etiqueta, invitaciones y transferencia de propiedad son funciones RPC en SQL.
- **Las rutas de API son adaptadores delgados**: validan sesión, rol y límite de peticiones, y delegan en `lib/` (por ejemplo, `/api/whatsapp/send` y `/api/v1/messages` comparten `sendMessageToConversation`).
- **Los webhooks no tienen vista**: entran por una ruta, pasan a un núcleo común de ingesta y desde ahí se reparten a automatizaciones, flujos, IA y webhooks salientes.

## 4.2 Diagrama de capas



Proveedores de IA

Hay dos clientes de Supabase en el servidor:

- `src/lib/supabase/server.ts`: usa la sesión del usuario (RLS activo). Lo usan las rutas que actúan en nombre de alguien.
- `supabaseAdmin` (`src/lib/*/admin-client.ts`): usa la `SUPABASE_SERVICE_ROLE_KEY` y se salta RLS. Lo usan webhooks, motores de flujos y automatizaciones, IA y la API pública, que no tienen sesión de usuario. Esas rutas filtran siempre por `account_id` a mano.

4.3 Recorrido de una petición real: un cliente escribe por WhatsApp y responde la IA

| # | Archivo                                                                                             | Qué pasa                                                                                                                                                                                                   |
|---|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Meta → <code>POST /api/whatsapp/webhook</code> → <code>src/app/api/whatsapp/webhook/route.ts</code> | Llega el evento.<br><code>verifyMetaWebhookSignature()</code> ( <code>src/lib/whatsapp/webhook-signature.ts</code> ) valida la firma HMAC-SHA256 con <code>META_APP_SECRET</code> ; si falla, responde 401 |
| 2 | mismo archivo                                                                                       | Responde 200 a Meta al instante y programa el procesamiento con <code>after()</code> para que no se corte en serverless                                                                                    |
| 3 | mismo archivo                                                                                       | Busca la fila de <code>whatsapp_config</code> cuyo <code>phone_number_id</code> coincide, y así sabe a qué cuenta (empresa) pertenece el mensaje                                                           |
| 4 | <code>src/lib/inbound/ingest-core.ts</code> → <code>resolveContactAndConversation()</code>          | Encuentra o crea el contacto (deduplicación en <code>src/lib/contacts/dedupe.ts</code> ) y la conversación ( <code>account_id</code> , <code>contact_id</code> , <code>channel</code> )                    |
| 5 | <code>ingest-core.ts</code> → <code>recordAndDispatchMessage()</code>                               | Guarda el mensaje en <code>messages</code> . Supabase Realtime lo empuja a la bandeja abierta ( <code>src/hooks/use-realtime.ts</code> → <code>src/components/inbox/message-thread.tsx</code> )            |
| 6 | <code>src/lib/flows/engine.ts</code> → <code>dispatchInboundToFlows()</code>                        | Si un flujo activo captura el mensaje, gana el flujo y la IA no responde                                                                                                                                   |
| 7 | <code>src/lib/automations/engine.ts</code> → <code>runAutomationsForTrigger()</code>                | Ejecuta las automatizaciones cuyo disparador coincide                                                                                                                                                      |
| 8 | <code>src/lib/ai/auto-reply.ts</code> → <code>dispatchInboundToAiReply()</code>                     | Comprueba IA activa, canal habilitado, que no haya agente asignado y el tope de respuestas por conversación                                                                                                |

| #  | Archivo                                                                                                                     | Qué pasa                                                                                                                                                                                                                                           |
|----|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 9  | <code>src/lib/ai/knowledge.ts</code> → <code>retrieveKnowledge()</code>                                                     | Busca fragmentos relevantes: semántica (pgvector) si hay clave de embeddings, si no texto completo                                                                                                                                                 |
| 10 | <code>src/lib/ai/generate.ts</code> → <code>generateReply()</code> → <code>src/lib/ai/providers/&lt;proveedor&gt;.ts</code> | Arma el <i>prompt</i> ( <code>src/lib/ai/defaults.ts</code> ) y llama al proveedor con la clave descifrada de la cuenta. Si el modelo emite <code>[[HANDOFF]]</code> , se deriva a humano y se avisa ( <code>src/lib/ai/handoff-notify.ts</code> ) |
| 11 | <code>src/lib/channels/router.ts</code> → <code>sendChannelText()</code>                                                    | Elige el emisor según <code>conversation.channel</code>                                                                                                                                                                                            |
| 12 | <code>src/lib/flows/meta-send.ts</code> → <code>src/lib/whatsapp/meta-api.ts</code>                                         | Envía por Graph API y guarda el mensaje saliente                                                                                                                                                                                                   |
| 13 | <code>src/lib/webhooks/deliver.ts</code> → <code>dispatchWebhookEvent()</code>                                              | Notifica <code>message.received</code> a los webhooks salientes registrados por la cuenta                                                                                                                                                          |

El camino inverso (un agente responde a mano desde la bandeja) es: `message-composer.tsx` → `fetch('/api/whatsapp/send')` → `src/app/api/whatsapp/send/route.ts` (sesión, límite de 60/min, cuenta) → `sendMessageToConversation()` en `src/lib/whatsapp/send-message.ts` → `meta-api.ts`.

**Limitación real:** ese camino solo funciona para WhatsApp. `sendMessageToConversation()` exige el teléfono del contacto, y los contactos de Messenger y Telegram no tienen. En esos canales la bandeja no puede responder a mano: solo responden la IA y las automatizaciones (que sí pasan por `channels/router.ts`).

## 5. Decisiones de diseño

### 5.1 Autenticación

- Supabase Auth con correo y contraseña (`src/app/(auth)/`). La sesión vive en cookies (`@supabase/ssr`).
- `src/middleware.ts` refresca la sesión en cada petición, copia las cookies renovadas a las redirecciones y manda a `/login` a quien no tenga sesión en `/dashboard`, `/inbox`, `/contacts`, `/pipelines`, `/broadcasts`, `/automations` y `/settings`. `/flows`, `/agents` y `/notifications` no están en esa lista, pero sus datos están protegidos igual por RLS y por las rutas de API.
- Rutas de API del panel: `requireRole(min)` en `src/lib/auth/account.ts` carga usuario, cuenta y rol, y lanza 401/403.
- API pública `/api/v1`: claves `Bearer` generadas con `randomBytes(32)`, guardadas solo como hash SHA-256 y comparadas con `timingSafeEqual` (`src/lib/api-keys/keys.ts`), con `scopes` por clave.
- Webhooks entrantes: Meta con HMAC-SHA256 (`META_APP_SECRET`); Telegram con el `secret_token` por cuenta en la cabecera `X-Telegram-Bot-API-Secret-Token`.

### 5.2 Roles y permisos

Cuatro roles por cuenta (**owner** 4 > **admin** 3 > **agent** 2 > **viewer** 1), definidos igual en TypeScript (`src/lib/auth/roles.ts`) y en SQL (`is_account_member`). Se aplican en tres lugares: RLS, `requireRole()` en las rutas, y `useCan()` en la interfaz para ocultar o desactivar botones.

| Acción                                                                                           | Rol mínimo | Dónde se exige                                                    |
|--------------------------------------------------------------------------------------------------|------------|-------------------------------------------------------------------|
| Ver datos de la cuenta                                                                           | viewer     | RLS <code>*_select</code>                                         |
| Contactos, negocios, difusiones, automatizaciones, flujos, enviar mensajes                       | agent      | RLS <code>*_insert/update</code> ; límites en rutas               |
| Embudos y etapas, etiquetas, campos personalizados, plantillas de Meta                           | admin      | RLS (migración 017)                                               |
| Configurar WhatsApp/Messenger/Telegram, IA, base de conocimiento, claves de API, miembros, marca | admin      | <code>requireRole('admin')</code>                                 |
| Transferir la propiedad, eliminar la cuenta                                                      | owner      | <code>canTransferOwnership</code> / <code>canDeleteAccount</code> |

### 5.3 Multiempresa

- Cada registro nuevo crea su propia **cuenta** (`accounts`) y queda como **owner** (`trigger handle_new_user`). Una sola instalación atiende a varias empresas.
- Todas las tablas de negocio tienen `account_id` y RLS por cuenta. Las rutas con *service role* filtran por `account_id` de forma explícita.
- Cada cuenta guarda sus propias credenciales: WhatsApp (`whatsapp_config`), Messenger (`messenger_config`), Telegram (`telegram_config`), IA (`ai_config`), Google Sheets y su marca (nombre, logo y colores, migración 037).
- El webhook de WhatsApp enruta por `phone_number_id`, el de Messenger por `page_id` y el de Telegram por `secret_token`.

**Límite para alquilar a varias empresas:** el secreto para verificar la firma de Meta es **uno solo para toda la instalación** (`META_APP_SECRET`). Por eso todas las empresas tienen que conectar WhatsApp y Messenger **a través de la misma app de Meta del operador**. No hay *Embedded Signup* ni registro como *Tech Provider*: el alta de cada número se hace a mano.

### 5.4 Seguridad

| Medida                  | Implementación                                                                                                                                                             |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cifrado de credenciales | AES-256-GCM con <code>ENCRYPTION_KEY</code> ( <code>src/lib/whatsapp/encryption.ts</code> ); los textos antiguos en AES-256-CBC se migran a GCM la primera vez que se leen |
| Aislamiento de datos    | RLS en todas las tablas                                                                                                                                                    |
| Webhooks entrantes      | HMAC-SHA256 (Meta), <code>secret_token</code> (Telegram)                                                                                                                   |

| Medida                  | Implementación                                                                                                                                                                                                                                                  |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Webhooks salientes      | Firma HMAC ( <a href="#">src/lib/webhooks/sign.ts</a> ) y bloqueo de IPs privadas/SSRF ( <a href="#">src/lib/webhooks/ssrf.ts</a> )                                                                                                                             |
| Límite de peticiones    | Ventana deslizante <b>en memoria</b> ( <a href="#">src/lib/rate-limit.ts</a> ): envío 60/min, difusión 90/min, reacciones 120/min, invitaciones 30 y 10/min. Con varias instancias del servidor, cada una cuenta por separado                                   |
| Cabeceras               | HSTS, <a href="#">nosniff</a> , <a href="#">X-Frame-Options: DENY</a> , <a href="#">Referrer-Policy</a> , <a href="#">Permissions-Policy</a> (solo micrófono propio) y CSP en modo <b>Report-Only</b> , todavía sin bloquear ( <a href="#">next.config.ts</a> ) |
| Caché                   | <a href="#">/api/*</a> con <a href="#">no-store</a>                                                                                                                                                                                                             |
| <i>Prompt injection</i> | El <i>prompt</i> de sistema indica tratar los mensajes del cliente como contenido no confiable ( <a href="#">src/lib/ai/default.ts</a> )                                                                                                                        |
| Marca                   | Colores validados como hex estricto y logo solo <a href="#">https</a> en base de datos (migración <a href="#">037</a> )                                                                                                                                         |

## 5.5 Configuración

- Variables de entorno usadas por el código: [NEXT\\_PUBLIC\\_SUPABASE\\_URL](#), [NEXT\\_PUBLIC\\_SUPABASE\\_ANON\\_KEY](#), [SUPABASE\\_SERVICE\\_ROLE\\_KEY](#), [ENCRYPTION\\_KEY](#), [META\\_APP\\_SECRET](#), [NEXT\\_PUBLIC\\_SITE\\_URL](#), y opcionales [META\\_APP\\_ID](#), [AUTOMATION\\_CRON\\_SECRET](#), [ALLOWED\\_INVITE\\_HOSTS](#), [WHATSAPP\\_TEMPLATES\\_DRY\\_RUN](#), [AI\\_REQUEST\\_TIMEOUT\\_MS](#), [AI\\_CONTEXT\\_MESSAGE\\_LIMIT](#). Están explicadas en [.env.local.example](#) y en la [guía](#).
- Todo lo demás (canales, IA, marca, moneda) se configura por cuenta desde la interfaz y se guarda en la base de datos.
- Tareas programadas: [/api/automations/cron](#) y [/api/flows/cron](#), protegidas con [x-cron-secret](#). Las ejecuta un servicio externo (por ejemplo [cron-job.org](#)); el proyecto no trae un *scheduler* propio.

## 5.6 Reglas de negocio importantes

- **Prioridad al responder:** primero el flujo, después las automatizaciones y por último la IA. Los flujos solo corren en WhatsApp.
- **La IA se calla** si la conversación tiene un agente asignado, si ya se derivó a humano ([ai\\_autoreply\\_disabled](#)), si llegó al tope de respuestas por conversación (por defecto 3, máximo 20) o si el canal está apagado en la configuración de IA.
- **Derivación a humano:** si el modelo emite [\[\[HANDOFF\]\]](#), el cliente recibe el mensaje de derivación y se avisa al humano (notificación interna y, si está configurado, un número de WhatsApp). El mensaje por defecto de [DEFAULT\\_HANDOFF\\_MESSAGE](#) incluye **un nombre y un teléfono fijos** ("Max Patricio", +51 989 377 295); cada cuenta debe cambiarlo en Configuración → IA.
- **Contactos:** se deduplican por teléfono normalizado (migración [022](#)); los de Messenger y Telegram se identifican por [contact\\_channel\\_identities](#). Un mismo cliente en dos canales tiene dos conversaciones.
- **Difusiones:** solo con plantillas aprobadas por Meta. El navegador envía lotes de 10 destinatarios con 1 s de pausa ([src/hooks/use-broadcast-sending.ts](#)) y reintenta si recibe 429. La difusión **avanza mientras la pestaña está abierta**: no hay cola en el servidor.

- **Esperas de automatizaciones:** solo avanzan si alguien llama al cron.
- **Marca:** el nombre por defecto es "Agente TED" ([src/lib/brand.ts](#)); cada cuenta puede cambiar nombre, logo y colores.

## 6. Estructura de carpetas

```

wacrm/
├── .github/                CI (ci.yml), Dependabot, plantillas de issues/PR,
CODEOWNERS (heredados del original)
├── docs/                  Documentación (este archivo y los enlazados arriba)
├── public/
│   ├── inbox-doodle.svg   Fondo de la bandeja
│   └── opus/              Worker de grabación de voz (copiado, no se compila)
├── supabase/migrations/   001...037 – esquema completo; se ejecutan a mano en el
SQL Editor y en orden
├── src/
│   ├── middleware.ts      Sesión y redirecciones
│   ├── app/
│   │   └── layout.tsx     HTML raíz, fuente, script de tema anterior a la
hidratación
│   │   ├── (auth)/        login, signup, forgot-password
│   │   ├── (dashboard)/   dashboard, inbox, contacts, pipelines, broadcasts,
│   │   │                   automations, flows, agents, notifications, settings
│   │   ├── join/[token]/  Aceptar invitación de equipo
│   │   └── api/
│   │       └── whatsapp/   webhook, send, broadcast, media, react, templates,
config
│   │   ├── webhooks/meta-omni/ webhook de Messenger
│   │   ├── messenger/ telegram/ configuración (+ webhook de Telegram)
│   │   └── ai/             config, draft, playground, test, knowledge, google-
sheets
│   │   ├── automations/ flows/ CRUD, motor y cron
│   │   └── account/ invitations/ miembros, invitaciones, claves de API,
transferencia
│   │   └── v1/             API pública REST
│   ├── components/        Un directorio por módulo; ui/ (shadcn) y tremor/
(gráficos) son base genérica
│   ├── hooks/             use-auth, use-can, use-realtime, use-theme, use-
presence...
│   ├── lib/
│   │   ├── inbound/       Núcleo de ingesta compartido por los canales
│   │   ├── channels/      Router de envío por canal
│   │   ├── whatsapp/ messenger/ telegram/ Integración de cada canal
│   │   ├── ai/            Proveedores, RAG, auto-respuesta, derivación
│   │   ├── google/        Google Sheets
│   │   ├── flows/ automations/ Motores
│   │   ├── auth/ api-keys/ api/v1/ webhooks/ Seguridad y API pública
│   │   ├── contacts/ inbox/ dashboard/ storage/ supabase/
│   │   └── brand.ts themes.ts currency.ts rate-limit.ts ...
│   └── types/             Tipos de filas de la base de datos
└── next.config.ts         Cabeceras de seguridad y caché

```

```
|— vitest.config.ts  eslint.config.mjs  .prettierrc  components.json  
tsconfig.json  
|— .env.local.example      Plantilla de variables de entorno
```

## 7. Tamaño del proyecto

Líneas **no vacías** (incluyen comentarios, que en este código son abundantes), sin `node_modules`, `.next` ni el `worker` de Opus copiado.

| Parte                                                                                                  | Líneas        |
|--------------------------------------------------------------------------------------------------------|---------------|
| Páginas ( <code>app/(dashboard)</code> , <code>(auth)</code> , <code>join</code> , raíz)               | 6,180         |
| Rutas de API ( <code>app/api</code> , sin pruebas)                                                     | 7,931         |
| Componentes por módulo (sin <code>ui/</code> ni <code>tremor/</code> )                                 | 23,624        |
| Servicios <code>lib/</code> (sin pruebas)                                                              | 14,089        |
| Hooks, tipos y middleware                                                                              | 2,217         |
| CSS                                                                                                    | 298           |
| Migraciones SQL (37)                                                                                   | 4,314         |
| Configuración ( <code>next</code> , <code>vitest</code> , <code>eslint</code> , <code>postcss</code> ) | 179           |
| <b>Subtotal de código de producción</b>                                                                | <b>58,832</b> |
| Pruebas (59 archivos)                                                                                  | 7,415         |
| Componentes base generados ( <code>ui/</code> shadcn + <code>tremor/</code> )                          | 2,732         |
| Documentación Markdown ( <code>docs/</code> , README, CHANGELOG, CONTRIBUTING)                         | ~2,500        |

Reparto aproximado por módulo funcional (TypeScript sin pruebas):

| Módulo                                                  | Líneas |
|---------------------------------------------------------|--------|
| Flujos (editor visual + motor)                          | 8,818  |
| WhatsApp: webhook, envío, medios, plantillas            | 7,714  |
| Bandeja                                                 | 4,663  |
| IA, base de conocimiento, Google Sheets                 | 4,416  |
| Cuentas, equipo, invitaciones, autenticación, presencia | 4,339  |
| Automatizaciones                                        | 3,930  |
| Contactos                                               | 3,374  |
| Difusiones                                              | 3,308  |
| Panel y notificaciones                                  | 2,794  |

| Módulo                                  | Líneas |
|-----------------------------------------|--------|
| Omnicanal: ingesta, Messenger, Telegram | 2,777  |
| API pública y webhooks salientes        | 2,486  |
| Embudos de venta                        | 2,089  |
| Marca, temas y <i>layout</i>            | 1,755  |

**Aporte propio frente a la base:** desde la última fusión con el original ([274db1c](#)), [src/](#) y [supabase/](#) suman **9,720 líneas agregadas y 2,705 borradas** en 202 archivos. Una parte importante de esos cambios es traducción de textos al español, no funcionalidad nueva.